Movie Review Sentiment Analyzer

NLP Mini Project Report – Semester VII

1. INTRODUCTION

Online platforms collect millions of reviews every day for movies, products, restaurants and apps. Reading them one by one is impossible, yet companies and viewers want to know quickly whether the general opinion is positive or negative. Sentiment analysis is a Natural Language Processing (NLP) task that does this automatically by classifying a piece of text as positive or negative.

The Movie Review Sentiment Analyzer is an NLP system with two parts. The first part is a classical machine-learning baseline that uses TF-IDF features and Logistic Regression. The second part is a deep-learning model: DistilBERT, a pretrained transformer, fine-tuned on movie reviews. Both models are trained on the IMDB review dataset and compared using accuracy, F1 score and confusion matrices. The best model is then deployed in a simple Gradio web application where a user can type any review and instantly see the predicted sentiment with a confidence score.

The project also shows why transformer models usually outperform word-count methods: they read words in context, so a phrase such as “not good” is understood as negative even though it contains the word “good”.

2. OBJECTIVES

The main objective is to build a sentiment analysis system that classifies movie reviews as positive or negative, and to compare a classical approach with a transformer-based approach.

The specific objectives are:

- To collect and preprocess a labelled movie-review dataset (IMDB).
- To build a baseline classifier using TF-IDF and Logistic Regression.
- To fine-tune a pretrained DistilBERT model for the same task.
- To evaluate both models using accuracy, F1 score and confusion matrix.
- To compare the two approaches and explain the difference in performance.
- To deploy the final model as an interactive web demo using Gradio.

3. PROBLEM STATEMENT

Manual reading of large numbers of reviews is slow and inconsistent. Simple keyword-based methods fail when sentences contain negation, sarcasm or mixed opinions. Training a transformer from scratch needs huge data and computing power, which students rarely have.

Proposed Solutions

- **Too many reviews to read manually:** Build an automatic classifier that labels each review as positive or negative in milliseconds.
- **Keyword methods ignore context:** Use a pretrained transformer (DistilBERT) that understands the meaning of words in their sentence.
- **Limited computing power:** Fine-tune a small, fast model on a subset of the data using a free Google Colab GPU.
- **Need for a fair comparison:** Train a classical TF-IDF + Logistic Regression baseline and evaluate both models on the same test set.
- **Hard for non-technical users to try:** Provide a simple Gradio interface where any review can be typed and tested.

4. TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose
Python	Main programming language
Google Colab (T4 GPU)	Cloud notebook and free GPU for training
Hugging Face Datasets	Loading the IMDB dataset
Hugging Face Transformers	DistilBERT model, tokenizer and Trainer API
PyTorch	Deep-learning backend for the transformer
scikit-learn	TF-IDF, Logistic Regression and evaluation metrics

Tool / Technology	Purpose
Pandas and NumPy	Data handling and results table
Matplotlib	Confusion matrix plots
Gradio	Interactive web demo

5. SYSTEM ARCHITECTURE

The system has two parts. The training pipeline runs in Google Colab and produces two trained models. The demo application loads the fine-tuned model and predicts the sentiment of any text typed by the user.

5.1 Overall Architecture

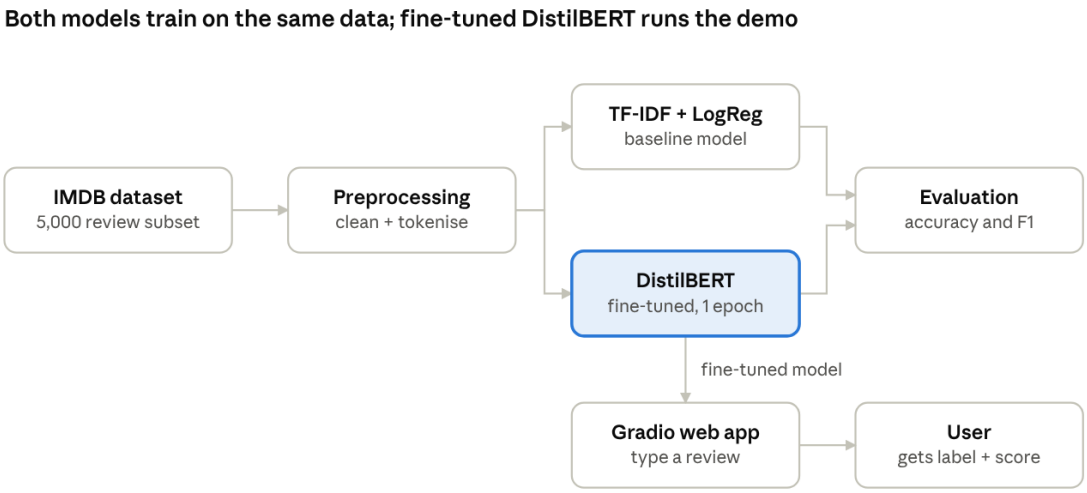


Figure 5.1: System architecture. Both models learn from the same preprocessed reviews and are scored on the same test set; the fine-tuned DistilBERT model is the one served in the Gradio app.

5.2 User Layer

- **User:** Types a movie review in the web app and receives the predicted sentiment with a confidence score.
- **Developer:** Trains and evaluates both models offline in the Colab notebook.

5.3 Application Layer

The application is a Gradio interface with one text box for input and a label output that shows the probability of Positive and Negative sentiment. It runs directly from the Colab notebook and can be shared through a temporary public link.

5.4 Data Preprocessing Layer

For the baseline model the text is cleaned in these steps:

- Convert text to lowercase.
- Remove HTML tags such as line-break tags that appear in IMDB reviews.
- Remove punctuation, digits and special characters.
- Remove English stopwords during TF-IDF vectorisation.

For DistilBERT the raw text is passed to the model's own tokenizer, which splits words into sub-word tokens, truncates every review to 128 tokens and pads shorter ones.

5.5 Baseline Model Layer (TF-IDF + Logistic Regression)

TF-IDF (Term Frequency – Inverse Document Frequency) converts each review into a numeric vector. A word gets a high score if it appears often in a review but rarely across all reviews, so common words matter less. The vocabulary is limited to the 20,000 most important words. Logistic Regression then learns a weight for each word and predicts the probability of the positive class.

5.6 Deep-Learning Layer (DistilBERT)

DistilBERT is a smaller, faster version of BERT that keeps about 97% of BERT's language understanding while being roughly 40% smaller. It is a transformer encoder pretrained on a large English text corpus. For this project a classification layer with two outputs (negative, positive) is added on top of the encoder, and the whole model is fine-tuned on the review data.

Stage	Description
Input	Review text, tokenised to 128 tokens
Embedding layer	Converts tokens into vectors with position information
Transformer encoder	6 layers of self-attention and feed-forward blocks
Classification head	Linear layer with 2 outputs
Output	Softmax probability of Negative and Positive

5.7 Training Setup

The models are trained on 4,000 randomly selected training reviews and tested on 1,000 randomly selected test reviews from IMDB, using a fixed random seed of 42 so that the results can be reproduced. DistilBERT is fine-tuned for 1 epoch with a batch size of 16 and a maximum sequence length of 128. The Hugging Face Trainer handles the optimiser and the training loop. Training runs on the free T4 GPU in Google Colab.

6. NLP MODEL IMPLEMENTATION

6.1 Dataset Collection and Preprocessing

The project uses the IMDB Large Movie Review dataset (stanfordnlp/imdb on Hugging Face). It contains 50,000 English movie reviews, split equally into 25,000 for training and 25,000 for testing, with each review labelled positive (1) or negative (0). The classes are perfectly balanced, so accuracy is a fair metric. To keep training fast, a random subset of 4,000 training reviews and 1,000 test reviews is used.

Item	Value
Dataset	IMDB movie reviews
Classes	Positive, Negative
Training samples used	4,000
Test samples used	1,000
Language	English

6.2 Model Descriptions

- **Model 1 – TF-IDF + Logistic Regression:** A fast and simple baseline. It treats each review as a bag of words and ignores word order.
- **Model 2 – DistilBERT:** A transformer that uses self-attention to understand how each word relates to the other words in the review, so it can handle negation and context.

6.3 Training and Evaluation Metrics

The following metrics are used on the 1,000 unseen test reviews:

- **Accuracy:** The share of reviews classified correctly.
- **F1 score:** The balance between precision and recall for the positive class.
- **Confusion matrix:** Shows true positives, true negatives, false positives and false negatives.

6.4 Model Testing and Evaluation

After training, both models predict the labels of the same held-out test set. Their metrics are placed side by side in a results table, and the confusion matrix of each model is plotted. The evaluation criteria are:

- **Prediction quality:** Accuracy and F1 score on unseen reviews.
- **Error pattern:** Which kind of mistakes each model makes, seen in the confusion matrix.
- **Context handling:** Whether the model correctly handles negation, such as "not good".
- **Speed:** Training and prediction time of each model.
- **Usability:** Whether the model works correctly on new reviews typed into the demo.

7. IMPLEMENTATION STEPS

7.1 Installing Libraries

```
!pip install -q transformers datasets gradio scikit-learn accelerate
```

7.2 Loading Data and Cleaning Text

```
ds = load_dataset("stanfordnlp/imdb")
train = ds["train"].shuffle(seed=42).select(range(4000))
test = ds["test"].shuffle(seed=42).select(range(1000))

def clean(t):
    t = re.sub(r"<.*?>", " ", t.lower())
    return re.sub(r"[^a-z\s]", " ", t)
```

7.3 Baseline: TF-IDF + Logistic Regression

```
vec = TfidfVectorizer(stop_words="english", max_features=20000)
lr = LogisticRegression(max_iter=1000).fit(vec.fit_transform(Xtr),
train["label"])
pred = lr.predict(vec.transform(Xte))
```

7.4 Fine-tuning DistilBERT

```
tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2)
args = TrainingArguments(output_dir="out", num_train_epochs=1,
                         per_device_train_batch_size=16)
trainer = Trainer(model=model, args=args, train_dataset=tr)
trainer.train()
```

7.5 Evaluation

```
out = trainer.predict(te)
bp = np.argmax(out.predictions, axis=1)
print(accuracy_score(test["label"], bp), f1_score(test["label"], bp))
```

7.6 Prediction Function for the Demo

```
def predict(text):
    inputs = tok(text, return_tensors="pt", truncation=True,
max_length=128).to(model.device)
    with torch.no_grad():
        p = torch.softmax(model(**inputs).logits, dim=-1)[0]
    return {"Negative": float(p[0]), "Positive": float(p[1])}
```

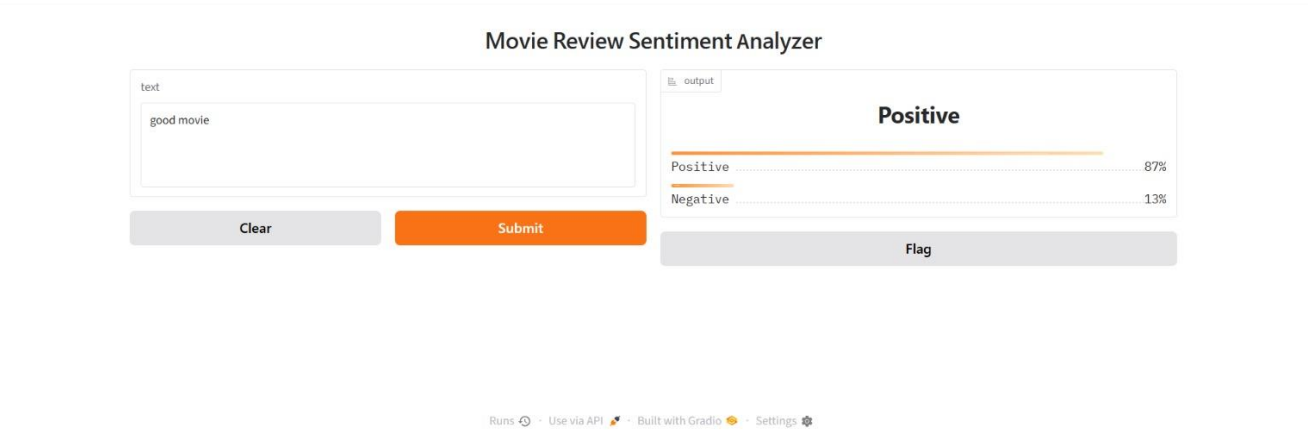
7.7 Gradio Web App

```
gr.Interface(fn=predict,
             inputs=gr.Textbox(lines=4, placeholder="Type a movie
review..."),
             outputs=gr.Label(num_top_classes=2),
             title="Movie Review Sentiment Analyzer").launch(share=True)
```

8. RESULTS

Replace every bracketed item below with the real output from your Colab notebook. Use your own numbers only.







9. FUTURE SCOPE

- **Train on the full dataset:** Use all 25,000 training reviews and more epochs for higher accuracy.
- **Multi-class sentiment:** Add a neutral class or a 1–5 star rating prediction.

- **Regional languages:** Extend the system to Hindi, Marathi or code-mixed Hinglish reviews with multilingual models such as mBERT or XLM-R.
- **Model comparison:** Compare with other transformers such as BERT, RoBERTa and LSTM models.
- **Explainability:** Highlight the words that influenced the prediction using LIME or SHAP.
- **Deployment:** Host the application permanently on Hugging Face Spaces.

10. CONCLUSION

The Movie Review Sentiment Analyzer demonstrates how NLP can turn large amounts of unstructured text into useful information. A classical TF-IDF and Logistic Regression baseline and a fine-tuned DistilBERT transformer were trained on the IMDB dataset and compared using accuracy, F1 score and confusion matrices. The comparison shows the trade-off between a simple, fast method and a context-aware deep-learning model. The final model was deployed as an interactive Gradio application that predicts the sentiment of any review with a confidence score. The project covers the full NLP pipeline, from data preprocessing and model training to evaluation and deployment.

REFERENCES

1. A. Maas et al., “Learning Word Vectors for Sentiment Analysis”, ACL, 2011 (IMDB dataset).
2. V. Sanh et al., “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”, 2019.
3. J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2019.
4. Hugging Face Transformers documentation – huggingface.co/docs/transformers
5. scikit-learn documentation – scikit-learn.org
6. Gradio documentation – gradio.app/docs